Computer and Network Security — Lecture 09

Dr. Karim Lounis

Jan - Jun 2026

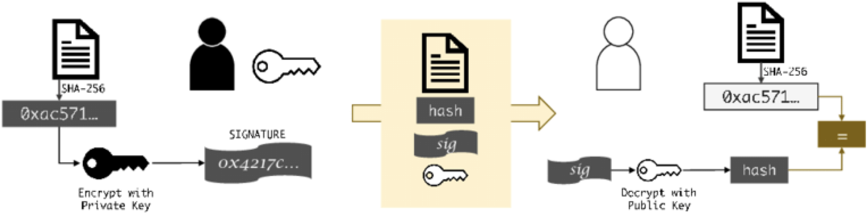
The National School of Artificial Intelligence
المدرسة الوطنية العليا للذكاء الاصطناعي

Digital Signature

Digital Signatures

Definition

Digital Signature. Is an encrypted hash value using a private key. It enforces authenticity, integrity of data, and non-repudiation. A signature is verified by decrypting it using the corrsponding public key.



Digital signatures can be generated using various public-key encryption schemes, e.g., RSA, ElGamal, Schnorr, Rabin, Lamport, Merkel, etc.

Another standardized algorithm is used: DSA (Digital Signature Algorithm), based on DLP, but considered less effecient than RSA.

Schnorr Digital Signature

Schnorr Digital Signature

Definition

Schnorr Signature. Is a digital signature scheme developed by Claus Schnorr in 1991. It relies on the discrete logarithm problem. It is efficient, simple, and generates short signatures. It works as follows:

- Define a cyclic subgroup $\mathcal{G}$ (of prime order q) over $\mathbb{Z}_p^*$, s.t. $q \mid p-1$. And let $g \in \mathbb{Z}_p^*$ be a generator of $\mathcal{G}$. Also, let $\mathcal{H}$ be a hash function.
- Party's keys are generated as: private key x is randomly selected from $\{1 \ldots q-1\}$ and the corresponding public key $y = g^x\ [p]$ is shared.
- A message m is signed as follows:
  - Randomly choose k from $\{1 \ldots q-1\}$,
  - Compute $r = g^k\ [p]$,
  - Compute $e = \mathcal{H}(r \mid\mid m)\ [q]$, where $\mid\mid$ is the concatenation, then
  - Compute $s = k - x \cdot e\ [q]$. The signature is (s, e).
- A signature (s, e) is verified as follows:
  - Compute $r' = g^s \cdot y^e\ [p]$,
  - Compute $e' = \mathcal{H}(r' \mid\mid m)$.
- If $e' = e$, the signature is valid. Otherwise, invalid.

Schnorr Digital Signature

Definition

Schnorr Signature. Is a digital signature scheme developed by Claus Schnorr in 1991. It relies on the discrete logarithm problems. It is efficient, simple, and g

Challenge (2.0 pts): Consider a case where Zoubir wants to use Schnorr signature. It uses the finite cyclic group $\mathcal{G}$, defined over $\mathbb{Z}^*_{503}$, and of order $q = 251$ and generator $g = 25$. The adopted hash function is the first 2 right-most digits of the `SHA512` function. Zoubir wants to sign the value **50** and send it to Massoud. Massoud verifies the signature. The private key of Zoubir is $x = 21$, the $||$ is an addition, and $k = 7$.

let

om

- Randomly choose k from $\{1 \dots q-1\}$,
- Compute $r = g^k \ [p]$,
- Compute $e = \mathcal{H}(r \ || \ m) \ [q]$, where $||$ is the concatenation, then
- Compute $s = k - x \cdot e \ [q]$. The signature is (s, e).

- A signature (s, e) is verified as follows:
  - Compute $r' = g^s \cdot y^e \ [p]$,
  - Compute $e' = \mathcal{H}(r' \ || \ m)$.
- If $e' = e$, the signature is valid. Otherwise, invalid.

Schnorr Digital Signature

Definition

Schnorr Signature. Is a digital signature scheme developed by Claus Schnorr in 1 ... d gen ...

Signing:

- The public key $y = 25^{21}\ [503] = 11$ is shared.
- Compute $r = 25^{7}\ [503] = 450$.
- Compute hash $e = \texttt{sha512}(450 + 50) = (42)_{16} = (66)_{10}$.
- Compute parameter $s = 7 - 21 \cdot 66\ [251] = 127$.
- Send $(s, e) = (127, 66)$.

Verifying:

- Compute $r' = 25^{127} \cdot 11^{66}\ [503] = 378 \cdot 97\ [503] = 450$.
- Compute $e' = \texttt{sha512}(450 + 50) = (42)_{16} = (66)_{10}$.
- Compare $e = 66$ and $e' = 66$. Signature is valid as $e' = e$.

Digital Signature Algorithm (DSA)

Digital Signature Algorithm (DSA)

Definition

DSA. Is a public-key cryptosystem and Federal Information Processing Standard (FIPS) for digital signatures, based on the discrete logarithm problem. DSA is a variant of the Schnorr & ElGamal signature schemes.

- NIST proposed it in 1991 and adopted it as FIPS186 in 1994.
- There have been various revisions (5).
- The newest spicification is FIPS186-5 (02/2023) and it forbids signing with classical DSA, but only to verify signature.
- OpenSSH announced that DSA is scheduled to be removed in 2025
- Originally, DSA (FIPS186-1) used SHA-1 as a hash function.
- Replaced by ECDSA (and EdDSA).
- ECDSA is employed in Bitcoin technology, but may get replaced by Schnorr signature (proposition).

Digital Signature Algorithm (DSA)

To use DSA, a set of parameters must be generated and shared among users:

- Choose an approved hash function $\mathcal{H}(\cdot)$ with output length $|\mathcal{H}|$ bits.
- Select a prime modulus p (1024-3072 bits)
- Select a prime divisor q of $p-1$.
- Select a generator g of the multiplicative subgroup of order q. We generally pose $g = h^{(p-1)/q}\ [p]$.
- The configuration parameters are (p, q, g).

For a given user (the signer):

- Choose an integer x randomly from $\{1 \ldots q-1\}$.
- Compute $y = g^x\ [p]$. Share the public key y and save the private key x.

Digital Signature Algorithm (DSA)

The signature is generated as follows (using the public key x):

1. Given a message m, choose a random integer $k \in \{1 \dots q-1\}$.
2. Compute $r = (g^k[p])[q]$ (if r=0, Goto 1).
3. Compute $s = (k^{-1} \cdot (\mathcal{H}(m) + x \cdot r))[q]$ (if s=0, Goto 1).
4. Append the signature (r, s) to the message m.

It is then verified as follows (using the public key y):

1. Verify that $0 < r < q$ and $0 < s < q$.
2. Compute $w = s^{-1}[q]$.
3. Compute $u_1 = \mathcal{H}(m) \cdot w[q]$.
4. Compute $u_2 = r \cdot w[q]$.
5. Compute $v = (g^{u_1} \cdot y^{u_2} \ [p])[q]$.
6. The signature is valid if and only if $v = r$.

Public Key Infrastructure (PKI)

Public Key Infrastructure (PKI)

Definition

PKI. Public key infrastructure is a set of roles, policies, hardware, software and procedures needed to create, manage, distribute, use, store and revoke digital certificates and manage public-key encryption.

PKI provides the following services:

- Register users or devices.
- Generate digital certificates.
- Renew digital certificates.
- Revoke digital certificates.
- Publish digital certificates.
- Publish the lists of revoked digital certificates.
- Play the trusted third party.

Public Key Infrastructure (PKI)

Definition

PKI. Public key infrastructure is a set of roles, policies, hardware, software and procedures needed to create, manage, distribute, use, store and revoke digital certificates and manage public-key encryption.

PKI is, generally, composed of:

- Certification authority (CA), responsible for issuing and signing digital certificates, and publishing signed lists of revoked certificates.
- Registration autheority (RA), responsible for generating the certificate and verifying the identity of the entity having requested a certificate (E.g., domain name owner).
- Storage authority (SA): Sotres certificates and revocation lists.

The term trusted third party (TTP) may also be used for CA.

Known CAs: DigiCert, Inc, VeriSign, Comodo Group, Let's Encrypt (free), GlobalSign, cPanel, and Entrust Datacard.

Digital Certificate

Definition

Digital Certificate. An electronic document that acts like a digital identity card in the online world. It **binds** a public key to an identity.

Certificate can be used for signing emails, verifyfing identity online, or accessing secure resources, secure resources (e.g., websites), authenticate emails, sign software for code verification, etc.

It contains the following information

- Subject: Identified entity (e.g., individual, device, domain, etc).
- Issuer: The CA that issued and signed the digital certificate.
- Public key of the entity (verification and encryption).
- Validity Period: The time for which certificate is valid.
- Digital signature: The CA's signature over the certificate.

There exists a standard format (defined by ITU) for digital certificates. It is known as **X.509**. It specifies its format and content.

Trusted Root CAs

Each CA is certified by another upper-level CA, . . ., up to a root CA.

Major browser vendors like Mozilla (Firefox), Google (Chrome), Apple (Safari), etc., maintain their own **trust stores**.

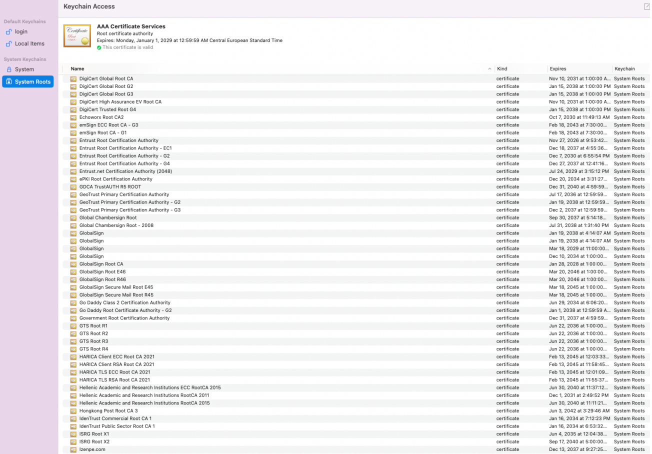
These stores contain certificates issued by well-known and reputable CAs that have undergone vetting and security audits. E.g.,:

- Entrust
- DigiCert
- Let's Encrypt (offers free certificates)
- Sectigo
- GlobalSign
- Comodo

Otherwise, your browser will likely display a warning message indicating that the website's certificate is signed by an untrusted authority.

Trusted Root CAs

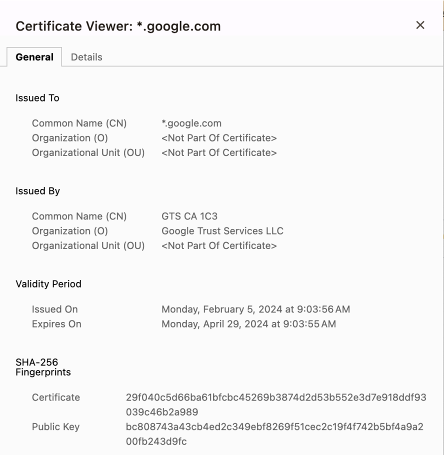
On Google Chrome web-browser, go to Setting>>Privacy and security>> Security>>Advanced>>Manage certificates:



Name	Kind	Expires	Keychain
DigiCert Global Root CA	certificate	Nov 10, 2031 at 1:00:00 A...	System Roots
DigiCert Global Root G2	certificate	Jan 15, 2038 at 1:00:00 PM	System Roots
DigiCert Global Root G3	certificate	Jan 15, 2038 at 1:00:00 PM	System Roots
DigiCert High Assurance EV Root CA	certificate	Nov 10, 2031 at 1:00:00 A...	System Roots
DigiCert Trusted Root G4	certificate	Jan 15, 2038 at 1:00:00 PM	System Roots
Echoworx Root CA2	certificate	Oct 7, 2030 at 11:49:13 AM	System Roots
emSign ECC Root CA - G3	certificate	Feb 18, 2043 at 7:30:00...	System Roots
emSign Root CA - G1	certificate	Feb 18, 2043 at 7:30:00...	System Roots
Entrust Root Certification Authority	certificate	Nov 27, 2026 at 9:53:42...	System Roots
Entrust Root Certification Authority - EC1	certificate	Dec 18, 2037 at 4:55:36...	System Roots
Entrust Root Certification Authority - G2	certificate	Dec 7, 2030 at 6:55:54 PM	System Roots
Entrust Root Certification Authority - G4	certificate	Dec 27, 2037 at 12:41:16...	System Roots
Entrust.net Certification Authority (2048)	certificate	Jul 24, 2029 at 3:15:12 PM	System Roots
ePKI Root Certification Authority	certificate	Dec 20, 2034 at 3:31:27...	System Roots
GDCA TrustAUTH R5 ROOT	certificate	Dec 31, 2040 at 4:59:59...	System Roots
GeoTrust Primary Certification Authority	certificate	Jul 17, 2036 at 12:59:59...	System Roots
GeoTrust Primary Certification Authority - G2	certificate	Jan 19, 2038 at 12:59:59...	System Roots
GeoTrust Primary Certification Authority - G3	certificate	Dec 2, 2037 at 12:59:59...	System Roots
Global Chambersign Root	certificate	Sep 30, 2037 at 5:14:18...	System Roots
Global Chambersign Root - 2008	certificate	Jul 31, 2038 at 1:31:40 PM	System Roots
GlobalSign	certificate	Jan 19, 2038 at 4:14:07 AM	System Roots
GlobalSign	certificate	Jan 19, 2038 at 4:14:07 AM	System Roots
GlobalSign	certificate	Mar 18, 2029 at 11:00:00...	System Roots
GlobalSign	certificate	Dec 10, 2034 at 1:00:00...	System Roots
GlobalSign Root CA	certificate	Jan 28, 2028 at 1:00:00...	System Roots
GlobalSign Root E46	certificate	Mar 20, 2046 at 1:00:00...	System Roots
GlobalSign Root R46	certificate	Mar 20, 2046 at 1:00:00...	System Roots
GlobalSign Secure Mail Root E45	certificate	Mar 18, 2045 at 1:00:00...	System Roots
GlobalSign Secure Mail Root R45	certificate	Mar 18, 2045 at 1:00:00...	System Roots
Go Daddy Class 2 Certification Authority	certificate	Jun 29, 2034 at 6:06:20...	System Roots
Go Daddy Root Certificate Authority - G2	certificate	Jan 1, 2038 at 12:59:59 A...	System Roots
Government Root Certification Authority	certificate	Dec 31, 2037 at 4:59:59...	System Roots
GTS Root R1	certificate	Jun 22, 2036 at 1:00:00...	System Roots
GTS Root R2	certificate	Jun 22, 2036 at 1:00:00...	System Roots
GTS Root R3	certificate	Jun 22, 2036 at 1:00:00...	System Roots
GTS Root R4	certificate	Jun 22, 2036 at 1:00:00...	System Roots
HARICA Client ECC Root CA 2021	certificate	Feb 13, 2045 at 12:03:33...	System Roots
HARICA Client RSA Root CA 2021	certificate	Feb 13, 2045 at 11:58:45...	System Roots
HARICA TLS ECC Root CA 2021	certificate	Feb 13, 2045 at 12:01:09...	System Roots
HARICA TLS RSA Root CA 2021	certificate	Feb 13, 2045 at 11:55:37...	System Roots
Hellenic Academic and Research Institutions ECC RootCA 2015	certificate	Jun 30, 2040 at 11:37:12...	System Roots
Hellenic Academic and Research Institutions RootCA 2011	certificate	Dec 1, 2031 at 2:49:52 PM	System Roots
Hellenic Academic and Research Institutions RootCA 2015	certificate	Jun 30, 2040 at 11:11:21...	System Roots
Hongkong Post Root CA 3	certificate	Jun 3, 2042 at 3:29:46 AM	System Roots
IdenTrust Commercial Root CA 1	certificate	Jan 16, 2034 at 7:12:23 PM	System Roots
IdenTrust Public Sector Root CA 1	certificate	Jan 16, 2034 at 6:53:32...	System Roots
ISRG Root X1	certificate	Jun 4, 2035 at 12:04:38...	System Roots
ISRG Root X2	certificate	Sep 17, 2040 at 5:00:00...	System Roots
Izenpe.com	certificate	Dec 13, 2037 at 9:27:25...	System Roots

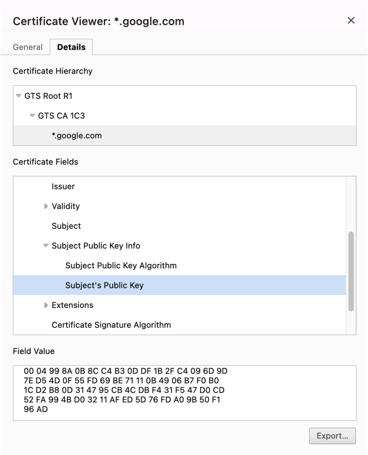
Digital Certificate

On your web-browser, you can check the digital certificate of secured domains. Example, the domain Google.com (2023):



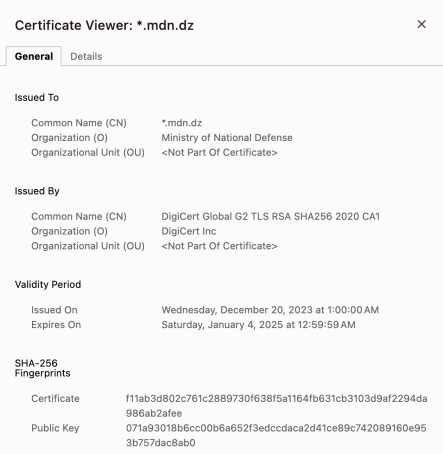
Digital Certificate

On your web-browser, you can check the digital certificate of secured domains. Example, the domain Google.com (2023):



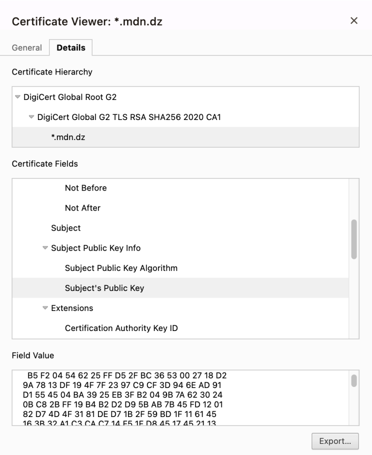
Digital Certificate

On your web-browser, you can check the digital certificate of secured domains. Example, the domain mdn.dz (2023):



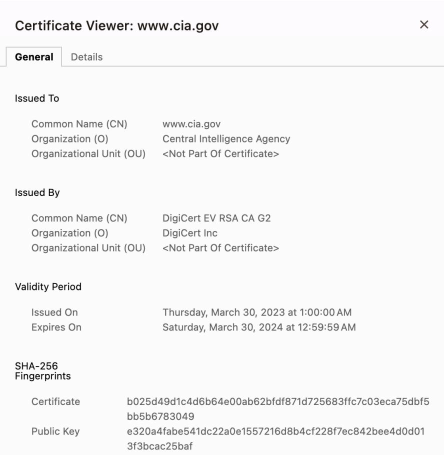
Digital Certificate

On your web-browser, you can check the digital certificate of secured domains. Example, the domain mdn.dz (2023):



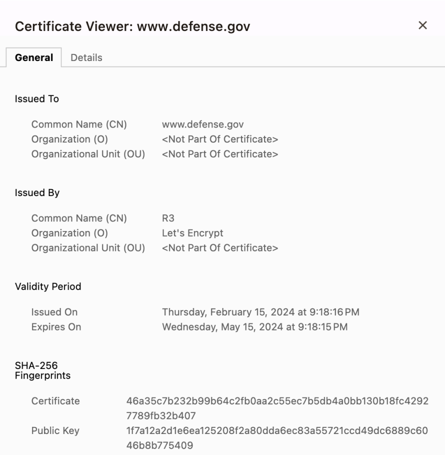
Digital Certificate

On your web-browser, you can check the digital certificate of secured domains. Example, the domain cia.gov (2023):



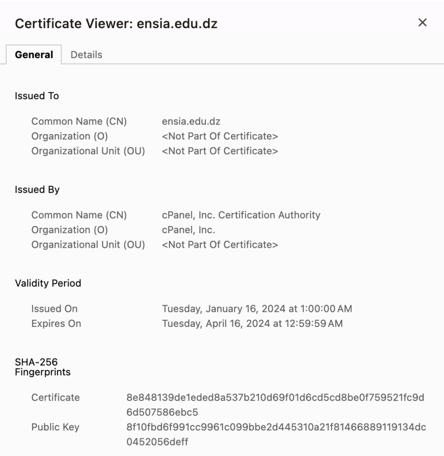
Digital Certificate

On your web-browser, you can check the digital certificate of secured domains. Example, the domain defense.gov (2023):



Digital Certificate

On your web-browser, you can check the digital certificate of secured domains. Example, the domain ensia.edu.dz (2023):



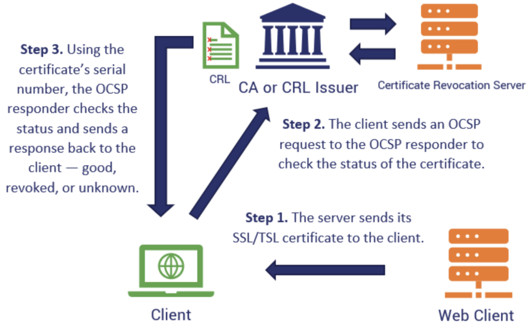
Digital Certificate Verification Steps

When you visit a website that uses HTTPS, the following happens:

1. The website's server sends to the client browser its certificate.
2. The browser inits the certificate chain checking process:
   - The certificate chain is a sequence of certificates, each signed by the certificate authority (CA) that issued the previous one. It typically ends with a trusted root certificate pre-installed in the browser.
   - The browser verifies the signature on each certificate in the chain using the public key of the issuing CA. This ensures that each certificate is genuine and hasn't been tampered with.
3. The browser checks the validity period of the certificate.
4. The browser also verifies the certificate's purpose.
5. The browser compares the website's domain name (extracted from the URL) with the Subject Name field in the certificate.

Digital Certificate (Checking Certificate Status: OCSP)

The browser may check a Certificate Revocation List (CRL) **or** use the Online Certificate Status Protocol (OCSP) to verify, in real-time, if the certificate has been revoked by the CA before its expiry date (Checking the validity).



CRL have limitations: Revocation latency, requires constant update of the list, and storage + communication overhead.

Put burden on the server: OCSP stapling instead of OCSP checking procedure.

OCSP Procedure

In standard OCSP, when your browser visits a website like whatever.dz, it needs to know: "Is this certificate still valid, or has it been revoked?"

- The Browser contacts the Certificate Authority (CA) directly.
- It asks the CA's OCSP Responder for the status.
- The CA responds.

The Downsides:

- **Privacy.** The CA now knows exactly which website you are visiting and when.
- **Performance.** It adds a "round-trip" delay (another DNS lookup and connection) before the page even starts loading.
- **Reliability.** If the CA's server is down, browser has to decide whether to "hard fail" (block the site) or "soft fail" (ignore security).

OCSP Stapling

With Stapling, the burden of proof shifts from the client (your browser) to the server (the website).

- The Server periodically contacts the CA itself, and asks, “Is my own certificate still good?”.
- The CA sends back a digitally signed, time-stamped response.
- The server caches the response.
- Then, when you connect to the server, the server “staples” that signed proof directly onto the initial TLS handshake.

PROMPT:

Given that root CA certificates are pre-installed and built in our browsers trust store (or OSs root store), is it possible that: (1) we use a malicous browser that a hacker made and inserted fake root CA certificates? (2) Can those certificates stored on the trust store of well-known broswers be poisoned?

Elliptic Curve Cryptography (ECC)

Elliptic Curve Cryptography (ECC)

Definition

Elliptic Curve Cryptography (ECC). It is a modern type of public-key cryptography that relies on the use of specific elliptic curve.

It has the following features:

- The currently most adopted type of cryptoraphy (e.g., EMV cards, epassport...).
- ECC provides equal security with smaller key sizes as compared to non-ECC algorithms (e.g., classical RSA, ElGamal...).
- Relies on the use of specific type of elliptic curves defined over a finite field.
- Allows faster computation, reduced memory, storage footprint, and required bandwidth, due to the use of smaller keys.

ECC Key Size	RSA Key Size
112	512
163	1024
192	1536
224	2048
256	3072
384	7680
512	15,360

Comparison of key sizes (bits)

Elliptic Curve Cryptography (ECC)

Definition

Elliptic Curve Cryptography (ECC). It is a modern type of public-key cryptography that relies on the use of specific elliptic curve.

It has the following features:

- The use of elliptic curves for cryptography was suggested, independently, by Neal Koblitz and Victor Miller in 1985. ECC started to be widely used after 2005.
- Quite a few of attacks developed for cryptography based on factorization and discrete logarithm do not work for the elliptic curves cryptography.
- Most governments have recommended to its governmental institutions to use mainly elliptic curve cryptography - ECC.

ECC Key Size	RSA Key Size
112	512
163	1024
192	1536
224	2048
256	3072
384	7680
512	15,360

Comparison of key sizes (bits)

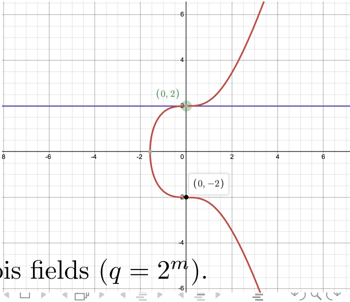
ECC — The Weiestrass Equation

Definition

Cryptographic Elliptic Curve. Is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b\ [q]$, together with a point at infinity $\mathcal{O}$ and a condition $4a^3 + 27b^2 \neq 0$ (for non-singularity). The curve parameters a and b are defined over a prime finite field $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, where q is a large prime number, or an integer of the form 2^m.

It has the following features:

- The curve is symmetric to x-axis.
- The curve goes to infinity. But we limit the curve up to a value $x = q$.
- If we draw a line on the curve, it will touch a maximum of 3 points.
- In cryptography, we can define elliptic curves over a prime finite field ($\mathbb{Z}_q$), and Galois fields ($q = 2^m$).



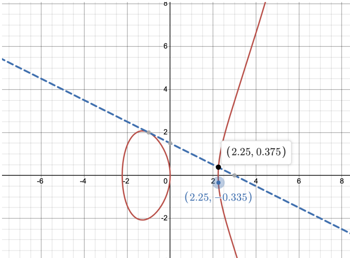
ECC — The Weiestrass Equation

Definition

Cryptographic Elliptic Curve. Is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b\ [q]$, together with a point at infinity $\mathcal{O}$ and a condition $4a^3 + 27b^2 \neq 0$ (for non-singularity). The curve parameters a and b are defined over a prime finite field $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, where q is a large prime number, or an integer of the form 2^m.

It has the following features:

- The curve is symmetric to x-axis.
- The curve goes to infinity. But we limit the curve up to a value $x = q$.
- If we draw a line on the curve, it will touch a maximum of 3 points.
- In cryptography, we can define elliptic curves over a prime finite field $(\mathbb{Z}_q)$, and Galois fields $(q = 2^m)$.



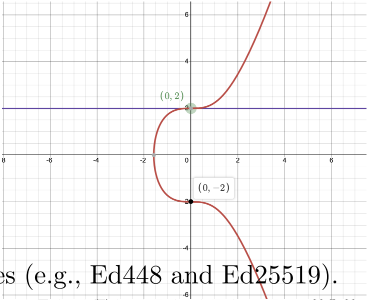
Elliptic Curve (EC) in Cryptography

Definition

Cryptographic Elliptic Curve. Is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b$, together with a point at infinity $\mathcal{O}$ and a condition $4a^3 + 27b^2 \neq 0$ (for non-singularity). The curve parameters a and b are defined over a prime finite field $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, where q is a large prime number, or an integer of the form 2^m.

It has the following features:

- Different a, b, and q, give different elliptic curves, one just better than the other.
- **Curve25519** is the most widely used. It is defined by the prime number $2^{255} - 19$, and uses the base point $x = 9$.
- Other cruves include: **NIST**'s (e.g., P-256, P-384, and P-512) and **Edwards**' curves (e.g., Ed448 and Ed25519).



Elliptic Curve (EC) in Cryptography

Definition

Cryptographic Elliptic Curve. Is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b$, together with a point at infi… …ve pa… …e q is…

It l…

Ouch!!!

In 2013, interest began to increase considerably when it was discovered that the NSA had potentially implemented a backdoor into the P-256 curve based Dual_EC_DRBG algorithm.[11] While not directly related,[12] suspicious aspects of the NIST's P curve constants[13] led to concerns[14] that the NSA had chosen values that gave them an advantage in breaking the encryption.[15][16]

> "I no longer trust the constants. I believe the NSA has manipulated them through their relationships with industry."
>
> — Bruce Schneier, The NSA Is Breaking Most Encryption on the Internet (2013)

Since 2013, Curve25519 has become the *de facto* alternative to P-256, being used in a wide variety of applications.[17] Starting in 2014, OpenSSH[18] defaults to Curve25519-based ECDH and GnuPG adds support for Ed25519 keys for signing and encryption.[19] The use of the curve was eventually standardized for both key exchange and signature in 2020.[20][21]

In 2017, NIST announced that Curve25519 and Curve448 would be added to Special Publication 800-186, which specifies approved elliptic curves for use by the US Federal Government.[22] Both are described in RFC 7748.[23] A 2019 draft of "FIPS 186-5" notes the intention to allow usage of Ed25519[24] for digital signatures. The 2023 update of Special Publication 800-186 allows usage of Curve25519.[25]

256, P-384, and P-512) and **Edwards'** curves (e.g., Ed448 and Ed25519).

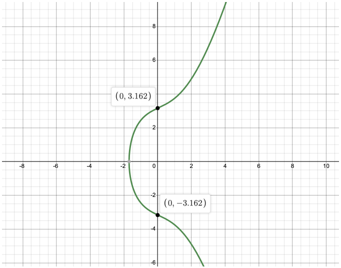
Elliptic Curve (EC) in Cryptography

Definition

Cryptographic Elliptic Curve. A CEC E is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b\ [q]$, together with a point at infinity $\mathcal{O}$ and a condition $4a^3 + 27b^2 \neq 0$ (for singularity). The curve parameters a and b are defined over a prime finite field $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, where q is a large prime number, or an integer of the form 2^m.

It has the following features:

- The set of points of an elliptic curve E, the point at infinity $\mathcal{O}$, and an operator "+" (addition of points) form an Abelian group., where $\mathcal{O}$ is the identity element.
- The point at infinity $\mathcal{O}$ represents the identity point. I.e., $\forall P \in E(\mathbb{F}_q) : P + \mathcal{O} = P$.
- Negation of a point $P = (x_P, y_P)$ in $E(\mathbb{F}_q)$ is $-P$ and is defined as $(x_P, -y_P)$.



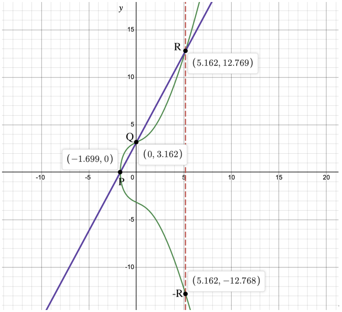
Elliptic Curve (EC) in Cryptography

Definition

Cryptographic Elliptic Curve. A CEC E is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b\ [q]$, together with a point at infinity $\mathcal{O}$ and a condition $4a^3 + 27b^2 \neq 0$ (for singularity). The curve parameters a and b are defined over a prime finite field $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, where q is a large prime number, or an integer of the form 2^m.

It has the following features:

- If we draw a line on the curve, it will touch a maximum of 3 points: P, Q, and R, such that: $P + Q = -R$, so, $-R + P = Q$.
- This addition is different from regular arithmetic addition because it considers the specific properties of the elliptic curve.
- The addition is closed, commutative, associative, and has an identity element ($\mathcal{O}$).



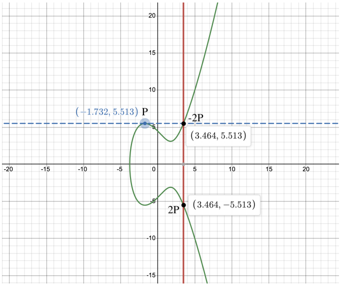
Elliptic Curve (EC) in Cryptography

Definition

Cryptographic Elliptic Curve. A CEC E is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b\ [q]$, together with a point at infinity $\mathcal{O}$ and a condition $4a^3 + 27b^2 \neq 0$ (for singularity). The curve parameters a and b are defined over a prime finite field $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, where q is a large prime number, or an integer of the form 2^m.

It has the following features:

- We can add a point to itself, a.k.a., point doubling: $P + P = 2P$.
- For a given point P, we draw the tangent line at that given point of the curve. The tangent crosses the curve on point $-2P$.
- Point doubling is a crucial building block for efficient scalar multiplication, another type of operation on elliptic curves.



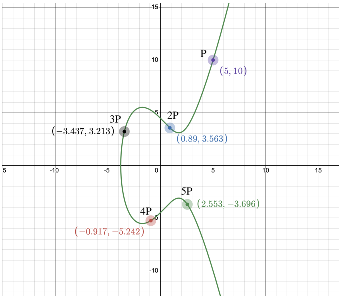
Elliptic Curve (EC) in Cryptography

Definition

Cryptographic Elliptic Curve. A CEC E is a set of points (x, y) defined by the solution to the equation $y^2 = x^3 + ax + b\ [q]$, together with a point at infinity $\mathcal{O}$ and a condition $4a^3 + 27b^2 \neq 0$ (for singularity). The curve parameters a and b are defined over a prime finite field $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, where q is a large prime number, or an integer of the form 2^m.

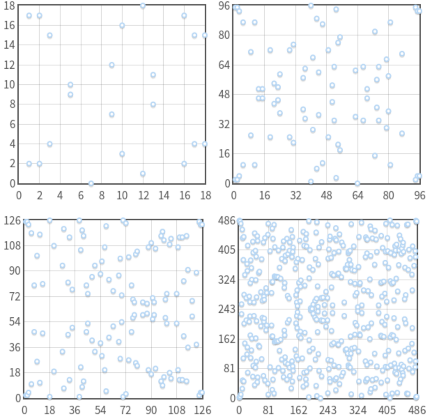
It has the following features:

- Point multiplication (scalar multiplication) over (E) can be computed as:
  $$n \cdot P = \underbrace{P + P + \ldots + P}_{\text{n times}}$$
- Point multiplicaiton happens to be very fast: To go from P to $n \cdot P$, you just add P to itself $(n-1)$ times. **However**, performing "division" is very slow. This is the basis of **ECDLP**.



Elliptic Curve (EC) in Cryptography

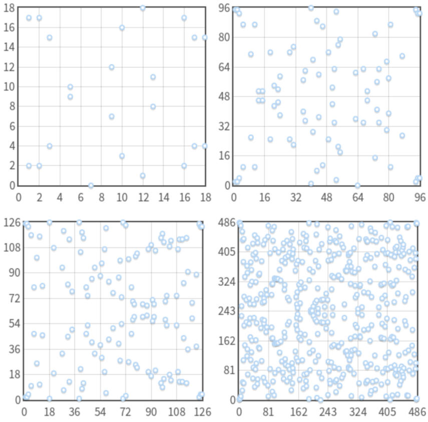
Remember, we work modulo q: $E : \quad y^2 = x^3 + ax + b\ [q]$



Given $G = n \cdot P$ (a dot in the graph), what was the value of n?

Elliptic Curve (EC) in Cryptography

Remember, we work modulo q: $E: \quad y^2 = x^3 + ax + b \; [q]$



Hasse's Theorem (Bound). The number of points on a cruve $E_q(a,b)$, denoted $\sharp E$ is given as: $q + 1 - 2\sqrt{q} \leq \sharp E \leq q + 1 + 2\sqrt{q}$ $\quad (\sharp E \approx q)$

Elliptic Curves Operations (addition)

Addition of points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ of an elliptic curve E can be computed as follows:

$$P_1 + P_2 = P_3 = (x_3, y_3)$$

Where:

$$x_3 = \lambda^2 - x_1 - x_2 \qquad \textbf{and} \qquad y_3 = \lambda \cdot (x_1 - x_3) - y_1$$

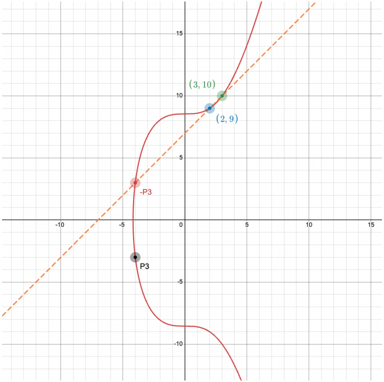
And

$$\lambda = \begin{cases} (y_2 - y_1)/(x_2 - x_1), & \text{if } P_1 \neq P_2, \\ (3 \cdot x_1^2 + a)/(2 \cdot y_1), & \text{if } P_1 = P_2 \end{cases}$$

Example. Consider an elliptic curve E defined as $y^2 = x^3 + 73$, and let $P_1 = (2, 9)$ and $P_2 = (3, 10)$. What is $P_3 = P_1 + P_2$ and $2 \cdot P_3$?

Elliptic Curves Operations (addition)

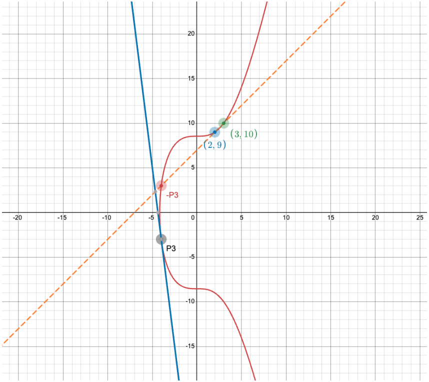
Example. Consider an elliptic curve E defined as $y^2 = x^3 + 73$, and let $P_1 = (2, 9)$ and $P_2 = (3, 10)$. What is $P_3 = P_1 + P_2$ and $2 \cdot P_3$?



$P_3 = (-4, -3)$ and $2 \cdot P_3 = (72, 611)$

Elliptic Curves Operations (addition)

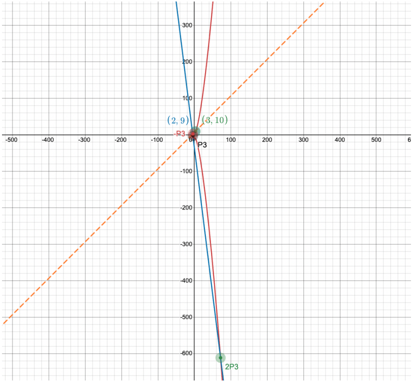
Example. Consider an elliptic curve E defined as $y^2 = x^3 + 73$, and let $P_1 = (2, 9)$ and $P_2 = (3, 10)$. What is $P_3 = P_1 + P_2$ and $2 \cdot P_3$?



$P_3 = (-4, -3)$ and $2 \cdot P_3 = (72, 611)$

Elliptic Curves Operations (addition)

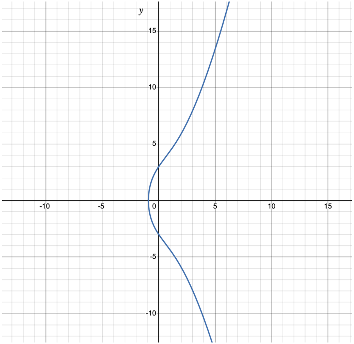
Example. Consider an elliptic curve E defined as $y^2 = x^3 + 73$, and let $P_1 = (2, 9)$ and $P_2 = (3, 10)$. What is $P_3 = P_1 + P_2$ and $2 \cdot P_3$?

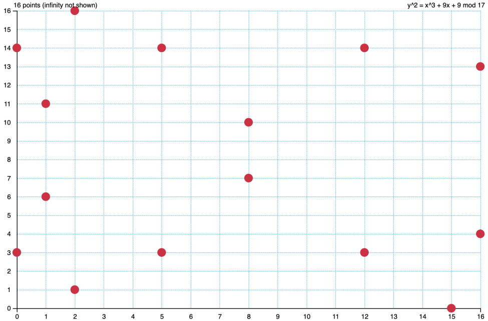


$P_3 = (-4, -3)$ and $2 \cdot P_3 = (72, 611)$

Elliptic Curves Operations (scalar multiplication)

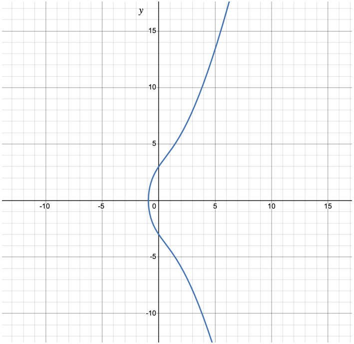
Example. Let $E_q(9,9)$, where $q = 17$, be a cryptographic elliptic curve defined over the finite field $\mathbb{Z}_{17}$. Compute $G = 9 \cdot (1,6)$.

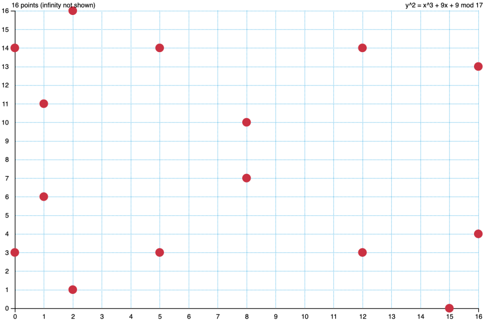




Elliptic Curves Operations (scalar multiplication)

Example. Let $E_q(9,9)$, where $q = 17$, be a cryptographic elliptic curve defined over the finite field $\mathbb{Z}_{17}$. Compute $G = 9 \cdot (1, 6)$.





The point G=9·(1, 6) is (5, 3)

Elliptic Curves Discrete Logarithm Problem (ECDLP)

The elliptic curve discrete logarithm problem is defined as follows:

Let E be an elliptic curve defined over $\mathbb{F}_q$, and let P and Q be two points of the curve $E(\mathbb{F}_q)$, such that: $Q = n \cdot P$ where $n \in \mathbb{F}_q$. Then. given P and Q, finding the value of n is computationally infeasible.

No efficient algorithm to compute discrete logarithm problem for elliptic curves is known and also no good general attacks.

Going from DLP-based protocols to ECDLP-based protocol consits of making the following changes:

- Assign a given message M to a point on a given elliptic curve E.
- Change modular multiplication to point addition on E.
- Change exponentiation to scalar multiplication on E.
- The resulting point represent the ciphertext.

Elliptic Curve Encryption and Decryption

Let Alice and Bob be two communication parties, where (n_A, P_A) and (n_B, P_B), are their elliptic curve public key pairs, respectively.

In ECC, encryption occurs as follows:

- Let M be a message to encrypt (requires a reversible encoding scheme).
- Encode the message into a point in the curve. Let this point be P_m.
- Choose a random integer k: the cipher point will be:

$$C_m = \{k \cdot G, P_m + (k \cdot P_B)\}$$

- Alice sends the ciphertext, which is the pair of points C_m to Bob.

Decryption occurs as follows:

- Compute $k \cdot G \cdot n_B$ with Bob's secret key n_B.
- Then subtract $(k \cdot G \cdot n_B)$ from the second point of the cipher point pair:

$$P_m + k \cdot P_B - (k \cdot G \cdot n_B) = P_m + k \cdot P_B - (k \cdot P_B) = P_m$$

- Retrieve the message M.

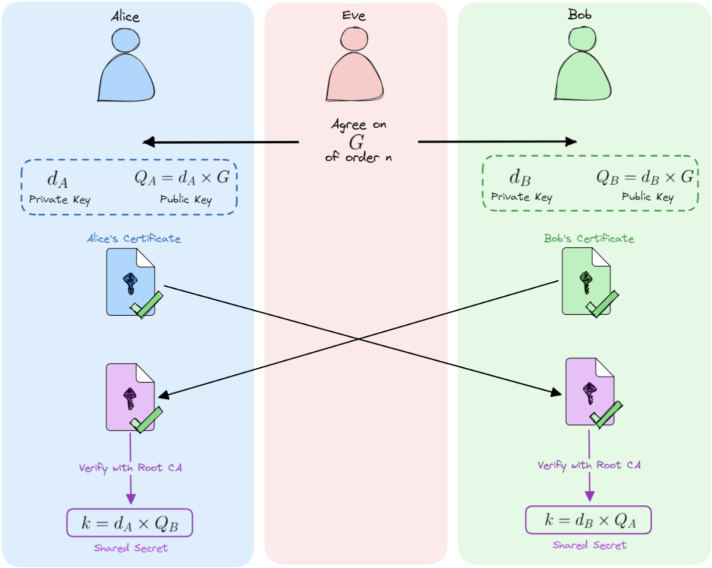
Elliptic Curve Diffie-Hellman Key Exchange

Elliptic Curve Cryptography Key Exchange (ECDH)

In ECC, the key generation is performed as follows:
First, we generate the global public elements:

- Select a cryptographic elliptic curve $E_q(a, b)$.
- Select a point G, the generator, which generates the cyclic group. I.e., every point in this group can be generated by repeat addition of this point a certain number of times. The order of the group is the number of points it can generate.
- User (Alice) key generation:
  - Select a private key n_A s.t. $n_A < n$ (where $n \cdot G = \mathcal{O}$)
  - Calculate the public key $P_A = n_A \cdot G$. (scalar multiplication)
- User (Bob) key generation:
  - Select a private key n_B s.t. $n_B < n$ (where $n \cdot G = \mathcal{O}$)
  - Calculate the public key $P_B = n_B \cdot G$. (scalar multiplication)
- Calculate the secret key by Alice: $k = n_A \cdot P_B$ (scalar multiplication)
- Calculate the secret key by Bob: $k = n_B \cdot P_A$ (scalar multiplication)

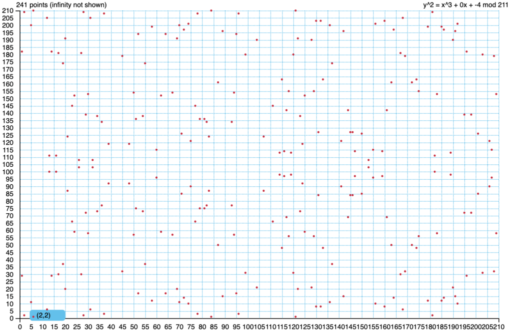
Elliptic Curve Diffie-Hellman Key Exchange



Elliptic Curve Diffie-Hellman Key Exchange (DIY)

Exercise. Let $E_q(0, -4)$, where $q = 211$, be a cryptographic elliptic curve defined over the finite field $\mathbb{Z}_{211}$. Let $G = (2, 2)$ be the generator.

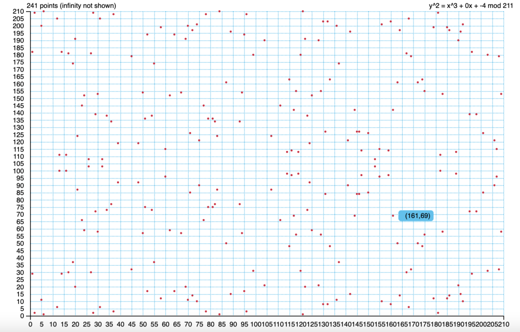
If Alice chooses $n_A = 121$ and Bob chooses $n_B = 203$. Apply ECDH to compute the secret "session" key.



Elliptic Curve Diffie-Hellman Key Exchange (DIY)

Exercise. Let $E_q(0, -4)$, where $q = 211$, be a cryptographic elliptic curve defined over the finite field $\mathbb{Z}_{211}$. Let $G = (2, 2)$ be the generator.

If Alice chooses $n_A = 121$ and Bob chooses $n_B = 203$. Apply ECDH to compute the secret "session" key. (Use online tools)

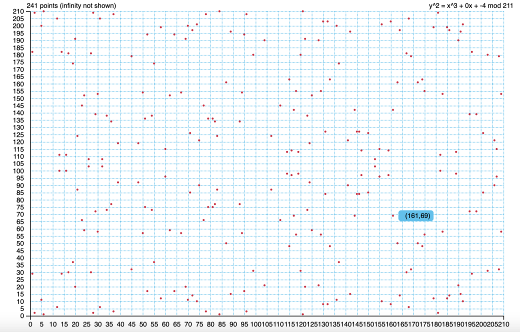


The secret key is: (161, 69).

Elliptic Curve Diffie-Hellman Key Exchange (DIY)

Exercise. Let $E_q(0, -4)$, where $q = 211$, be a cryptographic elliptic curve defined over the finite field $\mathbb{Z}_{211}$. Let $G = (2, 2)$ be the generator.

If Alice chooses $n_A = 121$ and Bob chooses $n_B = 203$. Apply ECDH to compute the secret "session" key. (Use online tools)



The secret key is: (161, 69). Encryption applis AES with K=161 or K=69.

- **End.**